

CTF WriteUp — re1

题目概述

题目给出两个文件：

- `Loader`
- `video.mp4`

整体来看，这是一道典型的 **多阶段加载 + 视频隐写 + 最终二进制还原** 题。

攻击链大致如下：

1. 分析 `Loader` 主程序逻辑
2. 发现程序内嵌了两个 base64 指针，但主流程只使用了其中一个“假入口”
3. 手工提取真正的 payload base64 并解码为 `pyc`
4. 分析该 Python 字节码的行为，发现它会从 `video.mp4` 中逐帧提取隐藏数据
5. 将视频中的黑白块恢复为二进制，并进行 `xor 0xAA`
6. 得到最终的 ELF payload
7. 分析 ELF 中的 MD5 数组与字符映射关系，恢复最终 flag

一、分析 Loader 主程序

首先对 `Loader` 进行静态分析。

可以发现它本体非常薄，核心逻辑大致如下：

1. 检查当前目录下 `video.mp4` 是否存在
2. 从程序内部读取一段 base64 字符串
3. base64 解码后写入文件 `stager.pyc`
4. 执行 `chmod 755 stager.pyc`
5. 运行：

```
1 python3 stager.pyc
```

也就是说，`Loader` 本身只是一个 **第一阶段加载器**，真正的逻辑不在 ELF 里，而在后续释放出来的 Python 字节码中。

二、关键误导点：.data 中的两个指针

继续查看 `Loader` 的 `.data` 段，可以发现这里有两个关键指针：

- `stager_pyc_base64 -> 0x4051`
- `payload_encoder_pyc_base64 -> 0x4058`

而 `main` 中实际只使用了前者，也就是：

```
1 | stager_pyc_base64
```

但是进一步查看对应内容后会发现：

- 0x4051 指向的并不是真正 payload
- 它实际指向的是字符串：

```
1 | delete
```

而真正的 base64 payload 存放在：

```
1 | 0x4058
```

这一步是题目的一个明显干扰点。

也就是说：

`main` 表面上使用的是第一个指针，但真正有效的 Python 载荷其实藏在第二个指针所指向的数据里。

三、提取真正的 Python Payload

将 0x4058 位置处的 base64 数据提取出来后，进行解码，可以得到：

```
1 | payload_real.pyc
```

对其做字符串分析，可以看到许多非常明显的特征，例如：

```
1 | video.mp4
2 | Payload_To_PixelCode_video.py
3 | PIL
4 | numpy
5 | imageio
6 | file_to_video
```

这说明这份 Python 字节码和视频逐帧处理密切相关。

从题目思路及还原结果来看，这个阶段的真实逻辑就是：

1. 逐帧读取 `video.mp4`
2. 每帧按 `8x8` block 划分
3. 对每个 block 计算平均亮度
4. 黑块记为 `1`
5. 白块记为 `0`
6. 每 8 bit 组合成 1 byte
7. 对每个字节再执行：

```
1 | xor 0xAA
```

最终得到隐藏的二进制数据。

四、视频隐写数据提取

对 `video.mp4` 进行处理时，可以发现视频分辨率为：

```
1 640 x 480
```

按 `8x8` block 取样后，每帧可得到大量 bit 数据。

按照题意还原流程：

- 黑块 -> `1`
- 白块 -> `0`
- 每 8 位拼成 1 字节
- 每个字节再异或 `0xAA`

恢复出来的结果文件开头如下：

```
1 7f 45 4c 46
```

也就是 ELF 魔数：

```
1 \x7fELF
```

说明视频中真正隐藏的是一个 **Linux ELF 可执行文件**。

因此可以得到第二阶段的 payload：

```
1 payload.bin
```

五、最终 ELF Payload 分析

对 `payload.bin` 继续做静态分析，可以发现它没有复杂控制流，主要逻辑是：

1. 将 `.data` 段中的指针数组拷贝到栈上
2. 输出提示信息
3. 实际上 flag 并不是直接明文存放，而是通过 **MD5 值数组** 间接表示

在字符串中可以看到一组 MD5：

```
1 8277e0910d750195b448797616e091ad
2 0cc175b9c0f1b6a831c399e269772661
3 4b43b0aee35624cd95b910189b3dc231
4 e358efa489f58062f10dd7316b65649e
5 f95b70fdc3088560732a5ac135644506
6 ...
```

同时程序中还有明显提示：

1	MD5
2	ASCII
3	flag

这说明解题思路是：

将这些 MD5 分别映射回单字符的 ASCII 明文，再按 `.data` 中数组指针的顺序组合起来。

六、MD5 到 ASCII 的映射

对这批 MD5 进行逐个匹配，可得到如下关系：

MD5	字符
8277e0910d750195b448797616e091ad	d
0cc175b9c0f1b6a831c399e269772661	a
4b43b0aee35624cd95b910189b3dc231	r
e358efa489f58062f10dd7316b65649e	t
f95b70fdc3088560732a5ac135644506	{
c81e728d9d4c2f636f067f89cc14862c	2
92eb5ffee6ae2fec3ad71c777531578f	b
c4ca4238a0b923820dcc509a6f75849b	1
8fa14cdd754f91cc6554c9e71929cce7	f
c9f0f895fb98ab9159f51fd0297e236d	8
336d5ebc5436534e61d16e63ddfca327	-
eccbc87e4b5ce2fe28308fd9f2a7baf3	3
cfcd208495d565ef66e7dff9f98764da	0
a87ff679a2f3e71d9181a67b7542122c	4
e4da3b7fbbce2345d7772b0674a318d5	5
e1671797c52e15f763380b45e841ec32	e
8f14e45fceeaa167a5a36dedd4bea2543	7
1679091c5a880faf6fb5e6087eb1b2dc	6
4a8a08f09d37b73795649038408b5f33	c
cbb184dd8e05c9709e5dcaedaa0495cf	}

七、按 .data 指针顺序重组 Flag

程序并不是直接顺序输出这些 MD5，而是将 `.data` 中的指针数组搬到栈上，再按照该数组的顺序取值。

恢复该顺序后，最终组合得到：

```
1 | dart{2ab1fb8a-b830-45e7-8830-66c7e3b3e05a}
```